

OPUS 的使用

采集和播放

由于之前有同学反映 Qt 的 QAudioInput 和 QAudioOutput 并不稳定, 故我们采集和播放采用更稳定的 SDL 来实现.

- SDL pro 文件添加库和依赖路径

```
INCLUDEPATH += $$PWD/SDL2-2.0.10/include
LIBS += $$PWD/SDL2-2.0.10/lib/x86/SDL2.lib
```

- SDL 添加头文件

```
extern "C" {
#include <SDL.h>
}
```

初始化 SDL

```
// 初始化 SDL
if (SDL_Init(SDL_INIT_AUDIO) < 0) {
    qDebug() << "Failed to initialize SDL: " << SDL_GetError() ;
    return;
}
```

主函数修改 SDL 的问题

```
#undef main
int main(int argc, char *argv[])
```

采集

- SDL 设置

```
// 设置音频规格
SDL_AudioSpec desiredSpec;
SDL_AudioSpec obtainedSpec;
SDL_memset(&desiredSpec, 0, sizeof(desiredSpec));
desiredSpec.freq = 48000; //采样率 48000Hz
desiredSpec.format = AUDIO_S16LSB; //16 位 小端存储
desiredSpec.channels = 1; //单声道
desiredSpec.samples = 960; // 回调函数 每 960 个点 回调一次
// 因为 1s 是 48000 点 那么 20ms 是 960 个点
desiredSpec.callback = audioCallback; //回调函数
desiredSpec.userdata = this; //回调函数传参
```

打开设备

```
// 打开录音设备 第一个为空意味着 默认设备
```

```

dev = SDL_OpenAudioDevice(NULL, 1, &desiredSpec, &obtainedSpec, 0);
if (dev == 0) {
    qDebug() << "Failed to open audio device: " << SDL_GetError() ;
    SDL_Quit();
    return ;
}

```

- 音频回调函数：作用为将采集到的数据 编码发送，此处稍后加编码

```

void SDLAudioRead::audioCallback(void *userdata, Uint8 *stream, int len)
{
    SDLAudioRead * audio = (SDLAudioRead *)userdata;
    if( len < 1920 ) return;

    QByteArray sendBuffer( (char*) stream, len); // 使用原缓冲区 避免拷贝

    //编码并发送 注意是多线程. 此并非主线程.
    audio->slot_sendAudioFrame( sendBuffer );
}

```

采集编码之后的数据通过信号发送到主线程.

- 开启采集

```

void slot_openAudio() {
    // 开始采集音频
    if( m_isOpen ) return;
    SDL_PauseAudioDevice(dev, 0);
    m_isOpen = true;
}

```

- 关闭采集

```

void slot_closeAudio() {
    // 停止采集音频
    if( !m_isOpen ) return;
    SDL_PauseAudioDevice(dev, 1);
    m_isOpen = false;
}

```

- SDL 关闭和回收

```

// 停止采集音频
SDL_PauseAudioDevice(dev, 1);
// 关闭音频设备并清理
SDL_CloseAudio();
SDL_Quit();

```

播放

- 设置音频

```
SDL_AudioSpec wantedSpec;
SDL_AudioSpec obtainedSpec;

SDL_zero(wantedSpec);
wantedSpec.freq = 48000;
wantedSpec.format = AUDIO_S16LSB;
wantedSpec.channels = 1;
wantedSpec.samples = 960;
wantedSpec.callback = audioCallback;
wantedSpec.userdata = this; //回调函数传参
```

- 打开设备

```
// 打开播放设备 第一个为空意味着 默认设备 , 第二个为 0 表示播放
dev = SDL_OpenAudioDevice(NULL, 0, &wantedSpec, &obtainedSpec, 0);
if (dev == 0) {
    qDebug() << "Failed to open audio device: " << SDL_GetError() ;
    SDL_Quit();
    return ;
}
```

- 采集到的数据添加到队列

```
void SDLAudioWrite::slot_playAudioFrame(QByteArray recvBuffer)
{
    if( !m_isOpen ) return;

    std::lock_guard<std::mutex> lck( m_mutex );
    m_audioQueue.emplace_back ( recvBuffer );
}
```

采集线程，播放线程和主线程不是一个线程，需要使用队列将数据中转。采集编码之后采用信号发送到主线程，主线程中将数据添加到队列，然后播放线程从队列取出并解码播放。

- 回调函数

```
void SDLAudioWrite::audioCallback(void *userdata, Uint8 *stream, int len)
{
    SDLAudioWrite * audio = (SDLAudioWrite *)userdata;

    memset( stream , 0 , len );
    if( !audio->m_audioQueue.empty() )
    {
        QByteArray recvBuffer = audio->m_audioQueue.front();
        {
```

```

        std::lock_guard<std::mutex> lck( audio->m_mutex );
        audio->m_audioQueue.pop_front();
    }
    //混音
    SDL_MixAudioFormat( stream , (uint8_t*)recvBuffer.data() , AUDIO_S16LSB
                        , recvBuffer.size() , 100);
}
}

```

- 开始播放

```

void slot_openAudio() {
    // 开始播放音频
    if( m_isOpen ) return;
    SDL_PauseAudioDevice(dev, 0);
    m_isOpen = true;
}

```

- 关闭播放

```

void slot_closeAudio() {
    // 停止播放音频
    if( !m_isOpen ) return;
    SDL_PauseAudioDevice(dev, 1);
    m_isOpen = false;
}

```

- 回收 关闭

```

// 停止播放音频
SDL_PauseAudioDevice(dev, 1);
// 关闭音频设备和 SDL
SDL_CloseAudio();
SDL_Quit();

```

界面类添加对象，并添加信号槽连接

```

m_audioRead = new SDLAudioRead;
m_audioWrite = new SDLAudioWrite;

connect( m_audioRead , SIGNAL( SIG_sendAudioFrame(QByteArray)) ,
        m_audioWrite , SLOT( slot_playAudioFrame(QByteArray)) );

```

OPUS 编解码

包含头文件, 添加库

pro 文件中添加

```
INCLUDEPATH += $$PWD/Opus-1.4/include
```

```
LIBS += $$PWD/Opus-1.4/lib/libopus.a -lssp
```

库文件 添加 libopus.a 以及 -lssp 这个库主要用于栈判断异常, 必须要有.

包含头文件

```
extern "C"{  
#include <opus/opus.h>  
}
```

因为是 C 的程序, 所以要加 extern "C"

编码器

- 定义编码器

```
OpusEncoder* encoder;
```

- 初始化

```
// 初始化 Opus 编码器和解码器
```

```
int err;
```

```
encoder = opus_encoder_create(48000, 1,  
                              OPUS_APPLICATION_VOIP, &err);
```

- 编码

```
unsigned char opusData[4096];
```

```
int nbBytes = opus_encode(audio->encoder, (const opus_int16*)(stream), 960, opusData,  
sizeof(opusData));
```

```
if (nbBytes < 0) {  
    qDebug() << "Opus encode error:" << opus_strerror(nbBytes);  
    return;  
}
```

opus_encode 是编码函数 第一个参数是上面定义的编码器, 第二个参数是要编码的缓冲区, 然后第三参数是采样点数, 然后第四参数是输出的缓冲区, 以及第五参数长度.

- 回收 销毁编码器

```
opus_encoder_destroy(encoder);
```

解码器

- 定义解码器

```
OpusDecoder* decoder;
```

- 初始化

```
int err;  
decoder = opus_decoder_create(48000, 1, &err);
```

- 解码

```
//解码  
opus_int16 decodedData[4096];  
int frameSizeDecoded = opus_decode(audio->decoder, (const uchar*)recvBuffer.data(),  
recvBuffer.size(), decodedData, sizeof(decodedData)/sizeof(decodedData[0]), 0);  
if (frameSizeDecoded < 0) {  
    return;  
}
```

解码函数 `opus_decode` 第一个参数 解码器 第二个参数要解码的缓冲区, 第三参数缓冲区大小, 第四参数输出的缓冲区, 第五参数缓冲区大小, 第六参数选项, 默认值, 表示内容丢失也要解码

- 回收 销毁编码器

```
opus_decoder_destroy(decoder);
```